

LAB-2

WATTANAPONG SUTTAPAK

Outline

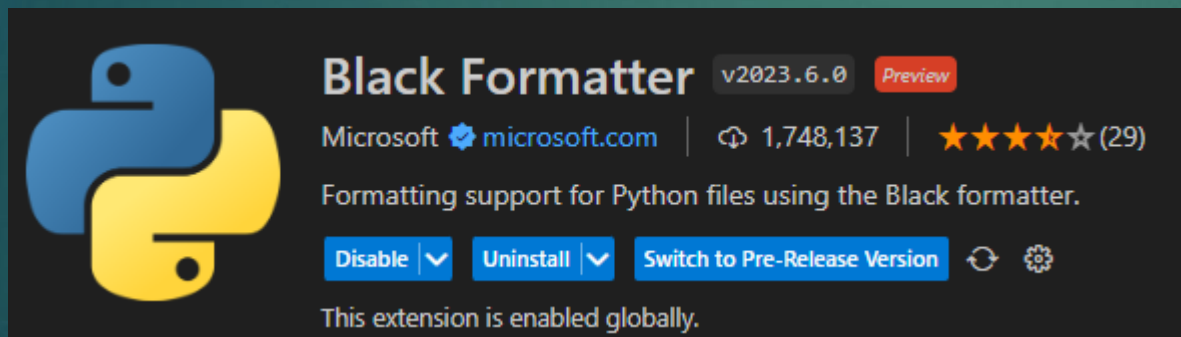
2

- ▶ Prerequisite
- ▶ Solving 8-puzzle game by
 - ▶ Depth-first Search
 - ▶ Breadth-first Search
- ▶ Assignment

Prerequisite

3

- ▶ Python Formatter (Extension)
- ▶ Python Class (Concept)
- ▶ Numpy (Library)



POWERFUL N-DIMENSIONAL ARRAYS

Fast and versatile, the NumPy vectorization, indexing, and broadcasting concepts are the de-facto standards of array computing today.

จัดการ Array n dimension
ได้อย่างทรงพลัง

NUMERICAL COMPUTING TOOLS

NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more.

มีฟังก์ชันมากมายในการคำนวณ
ทางตัวเลข

OPEN SOURCE

Distributed under a liberal [BSD license](#), NumPy is developed and maintained [publicly on GitHub](#) by a vibrant, responsive, and diverse [community](#).

ร่วมพัฒนาหรือแก้ไขโค้ดของ Numpy
เพิ่มเติมได้

INTEROPERABLE

NumPy supports a wide range of hardware and computing platforms, and plays well with distributed, GPU, and sparse array libraries.

รองรับการแปลงไปยัง Library อื่นเพื่อ
ใช้งานร่วมกันด้าน distributed
Computing

PERFORMANT

The core of NumPy is well-optimized C code. Enjoy the flexibility of Python with the speed of compiled code.

ทำงานเร็ว เพราะ library พื้นฐาน
optimize ด้วย C

EASY TO USE

NumPy's high level syntax makes it accessible and productive for programmers from any background or experience level.

ใช้ง่าย

NumPy

5

- ▶ Installation
 - ▶ `pip install numpy`
- ▶ Usage
 - ▶ `import numpy as np`
- ▶ Remarkable Function
 - ▶ Matrix Calculation ($a + b$, $a * b$, a / b for a and b are matrices)
 - ▶ Array Sorting

Matrix multiplication by NumPy

6

```
1 import numpy as np
2
3 # Create two matrices
4 matrix_a = np.array([[1, 2, 3],
5                      [4, 5, 6],
6                      [7, 8, 9]])
7
8 matrix_b = np.array([[9, 8, 7],
9                      [6, 5, 4],
10                     [3, 2, 1]])
11
12 # Matrix multiplication
13 result_multiplication = np.matmul(matrix_a, matrix_b)
14
15 # Matrix dot product
16 result_dot_product = np.dot(matrix_a, matrix_b)
17
18 # Display the results
19 print("Matrix A:")
20 print(matrix_a)
21
22 print("\nMatrix B:")
23 print(matrix_b)
24
25 print("\nMatrix Multiplication:")
26 print(result_multiplication)
27
28 print("\nMatrix Dot Product:")
29 print(result_dot_product)
```

Matrix A:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Matrix B:

```
[[9 8 7]
 [6 5 4]
 [3 2 1]]
```

Matrix Multiplication:

```
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

Matrix Dot Product:

```
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

```
# 2 rows, 2 columns, in 3 layers
a = np.random.rand(3,2,2)
b = np.random.rand(3,2,2)

# perform matrix multiplication
c = np.dot(a, b)
d = np.matmul(a,b) # a @ b

print(c.shape)    (3, 2, 3, 2)
print(d.shape)    (3, 2, 2)
```

- matmul for 2D or higher
- dot for vector or 2D

$$\begin{aligned} &\text{Ex. } [1,2,3] * [9,6,3] \\ &= 9 + 12 + 9 = 30 \end{aligned}$$

Sort by NumPy

7

```
1  import numpy as np
2
3  a = [2, 3, 9, 1, 4]
4
5  a.sort(reverse=False)
6  # return a as sort list ascending
7  print(a)
8
9  a = [2, 3, 9, 1, 4]
10 # return index of sorting array
11 indx = np.argsort(a)
12 print(indx)
13 print(a)
14
15 # convert list a to np array
16 print(np.array(a)[indx])
17
```

[1, 2, 3, 4, 9]

[3 0 1 4 2]
[2, 3, 9, 1, 4]

[1 2 3 4 9]

Python Class

8

ตัวแปรที่แสดงถึงความเป็นฟังก์ชันภายในไค้คลาส

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def set_name(self, name):
7         self.name = name
8
9     def set_age(self, age):
10        self.age = age
11
12    def display_info(self):
13        print(f"Name: {self.name}, Age: {self.age}")
```

constructor

```
15 # Example usage
16 person1 = Person("Sala Khunnawut", 61)
17
18 print("Initial Information:")
19 person1.display_info()
20
21 person2 = Person()
22 person2.set_name("ต่าย อรทัย")
23 person2.set_age(43)
24
25 person2.display_info()
```

Python Class: Overloading Constructor

9

```
1 class Person:
2     def __init__(self, name="", age=0):
3         self.name = name
4         self.age = age
5
6     def set_name(self, name):
7         self.name = name
8
9     def set_age(self, age):
10        self.age = age
11
12    def display_info(self):
13        print(f"Name: {self.name}, Age: {self.age}")
```

Note:

Python ไม่มี overloading method and overloading constructor
ใช้วิธีสร้าง method ที่รับ parameters หลายๆ ตัวแทน
ข้อเสีย การป้อน parameters ต้องป้อนตามลำดับ

```
16 # Example usage
17 person1 = Person("Sala Khunnawut", 61)
18
19 print("Initial Information:")
20 person1.display_info()
21
22 person2 = Person()
23 person2.set_name("ต่าย อรทัย")
24 person2.set_age(43)
25
26 person2.display_info()
27
28 person3 = Person(61)
29 person3.display_info()
```

การป้อนค่าผิดลำดับ

```
Initial Information:
Name: Sala Khunnawut, Age: 61
Name: ต่าย อรทัย, Age: 43
Name: 61, Age: 0
```

Recursion

10

- ▶ Uninformed Search usually solves problems by brute force.
- ▶ Brute force is the technique to search all possibility.
- ▶ 8-queen problem, Sudoku, and 8-puzzle problems, can solve using brute force technique.
- ▶ Brute force require recursive technique (Recursion) to call function itself.

8-Queen

11

		Q					
					Q		
							Q
Q							
			Q				
						Q	
				Q			
	Q						

1. Every Q placement needs the same function to check the rule.

- No more Q in one row
- No more Q in one column
- No more Q in one diagonal

1. Every Q placement can make mistake.

- Go back to previous step is necessary.
- This is the reason why we need recursion.

Type of Recursions

12

1. Linear Recursion
2. Binary Recursion
3. Multiple Recursion

Linear Recursion

13

1. Linear Recursion

เป็นการเขียนให้ recursive แบบง่าย สามารถเขียนเป็นแบบรูปธรรมดาได้ เช่น

โปรแกรมหาผลรวม ผลคูณ Factorial Fibonacci number

Loop

```
int product(int[] A, int n) {  
    int prod = 1;  
    for (int i = 0; i < n; i++) {  
        prod *= A[i];  
    }  
    return prod;  
}
```

Recursion

```
int product(int[] A, int n) {  
    if (n == 1)  
        return A[n - 1];  
    else  
        return A[n - 1] * product(A, n - 1);  
}
```

Linear Recursion

14

- ▶ ตัวอย่างที่ 1 factorial

Loop

```
int factorial(int n) {  
    int j = 1;  
    for (int i = 2; i <= n; i++) {  
        j = j * i;  
    }  
    return j;  
}
```

Recursion

```
int factorial(int n){  
    if ( n==1 ) return 1;  
    else return n*factorial(n-1);  
}
```

Linear Recursion

15

- ▶ ตัวอย่างที่ 1 factorial
 - ▶ วิธีการคิดแบบ recursive
 - ▶ $5!$ $\rightarrow \text{factorial}(5)$
 - ▶ $5! = 5 * 4!$ $\rightarrow 5 * \text{factorial}(4) \rightarrow n * \text{factorial}(n-1) \mid n = 5$
 - ▶ $4! = 4 * 3!$ $\rightarrow 4 * \text{factorial}(3) \rightarrow n * \text{factorial}(n-1) \mid n = 4$
 - ▶ $3! = 3 * 2!$
 - ▶ $2! = 2 * 1$

$n * \text{factorial}(n-1)$

Binary Recursion

16

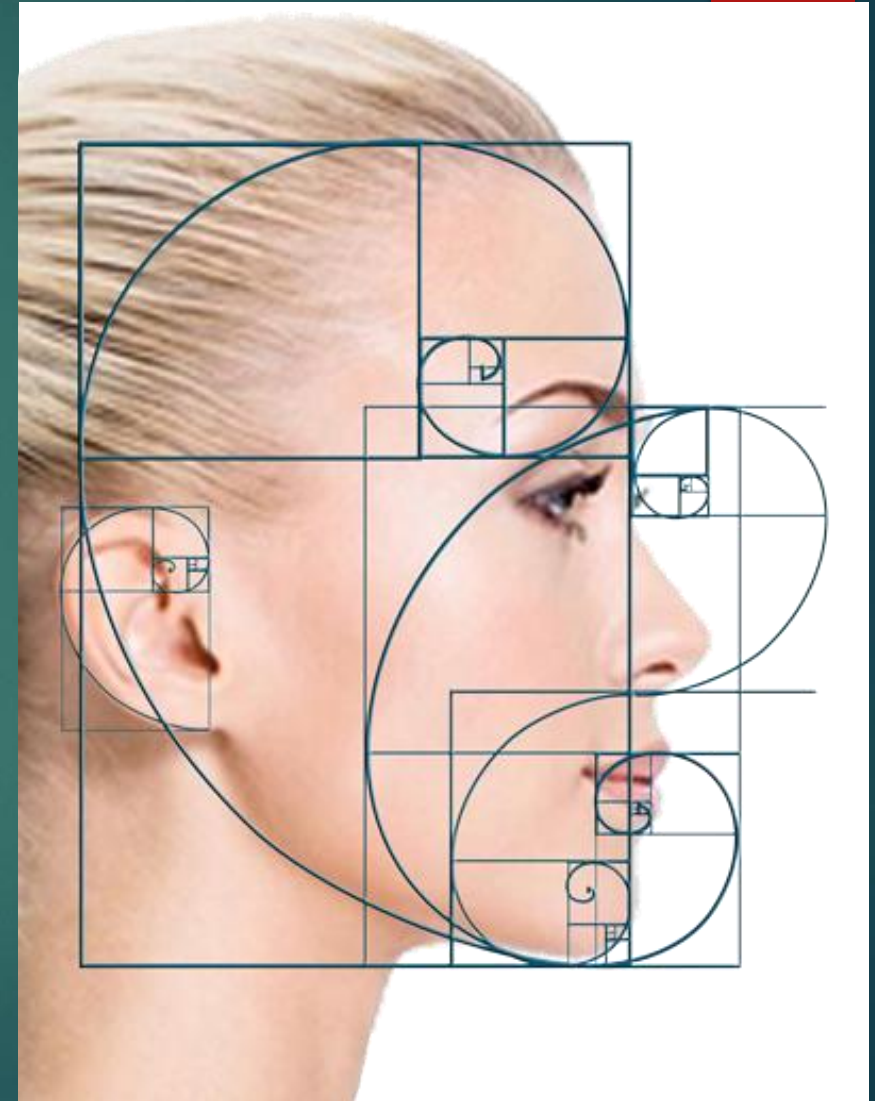
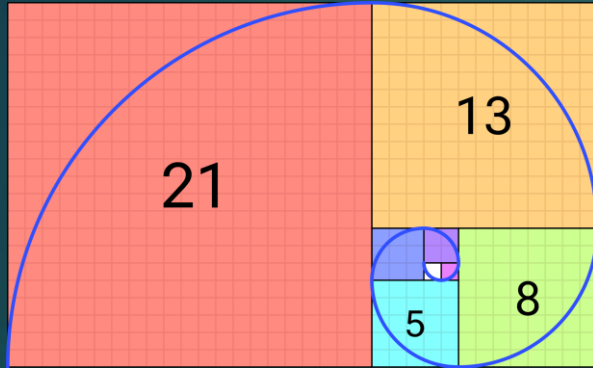
2. Binary Recursion

เป็นการเขียนฟังก์ชันเพื่อเรียกตัวเอง โดยทำการเรียกทีละ 2 recursive

- ตัวอย่างที่ 4 ลำดับเลข fibonacci
- หมายถึงลำดับเลขต่อไปนี้ $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots, \text{fib}(n-1) + \text{fib}(n-2)$

Fibonacci Magic

17



Binary Recursion

18

2. Binary Recursion

Loop

```
int fibonacci(int n) {  
    if (n == 1)  
        return 0;  
    else if (n == 2)  
        return 1;  
    else {  
        int a = 1, b = 1, i, c;  
        for (i = 3; i <= n; i++) {  
            c = a + b;  
            a = b;  
            b = c;  
        }  
        return a;  
    }  
}
```

Recursion

```
int fibonacci(int n) {  
    if (n == 1)  
        return 0;  
    else if (n == 2)  
        return 1;  
    else  
        return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Binary Recursion

19

- ▶ เปรียบเทียบจำนวนการคำนวณระหว่าง Iterative กับ Recursion

n	Iterative Algorithm	Recursive Algorithm
6	15	25
10	27	177
15	42	1,973
20	57	21,891
25	72	242,785
30	87	2,692,537

Multiple Recursion

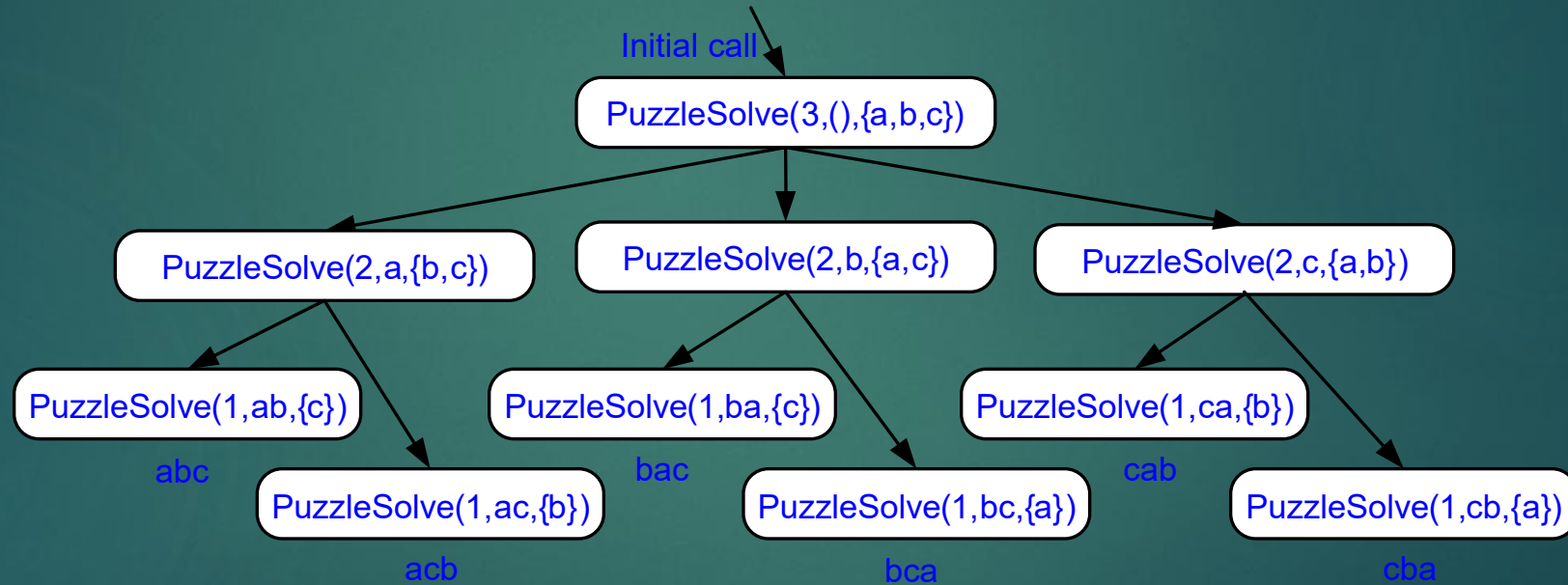
20

3. Multiple Recursion

- เป็นการเขียนฟังก์ชันเพื่อเรียกตัวเอง โดยทำการเรียกหลายๆ recursive
- ตัวอย่างโจทย์ที่ต้องใช้ Multiple Recursion เช่น Crossword, 8-Queen, 8-Puzzle, knapsack
- ตัวอย่าง ให้แสดงผลความน่าจะเป็นในการผสมตัวอักษรทั้งหมด จากตัวอักษรชุด
- เช่น ABC
- คำตอบที่ได้คือ ABC ACB BAC BCA CAB CBA

Multiple Recursion

21



8-Puzzle Problem

22

Start

1	2	3
	4	6
7	5	8

...

Goal

1	2	3
4	5	6
7	8	

Depth First Search

23

```
function depth_first_search(initial_state, goal_state):
    stack = [(initial_state, [])]
    visited = set()

    while stack is not empty:
        current_state, path = stack.pop()

        if current_state.puzzle is equal to goal_state.puzzle:
            return path

        visited.add(tuple(current_state.puzzle))

        for direction in ['up', 'down', 'left', 'right']:
            new_state = current_state.move(direction)

            if new_state is not null and tuple(new_state.puzzle) not in visited:
                stack.append((new_state, path + [direction]))

    return null
```

Depth First Search

24

```
class PuzzleState:
    function __init__(puzzle):
        set puzzle
        set size to length of puzzle
        set blank_position to index of 0 in puzzle

    function move(direction):
        new_state = create a copy of this state
        if direction is 'up' and new_state.blank_position >= 3:
            swap blank_position with the element above it
            decrement blank_position by 3
        elif direction is 'down' and new_state.blank_position < 6:
            swap blank_position with the element below it
            increment blank_position by 3
        elif direction is 'left' and new_state.blank_position % 3 is not 0:
            swap blank_position with the element to its left
            decrement blank_position by 1
        elif direction is 'right' and (new_state.blank_position + 1) % 3 is not 0:
            swap blank_position with the element to its right
            increment blank_position by 1
        else:
            return null
        return new_state
```

```
# Example usage:
initial_puzzle = [1, 2, 3, 4, 0, 5, 6, 7, 8]
goal_puzzle = [1, 2, 3, 4, 5, 6, 7, 8, 0]

initial_state = create an instance of PuzzleState with initial_puzzle
goal_state = create an instance of PuzzleState with goal_puzzle

solution_path = depth_first_search(initial_state, goal_state)

if solution_path is not null:
    print("Solution Path:", solution_path)
else:
    print("No solution found.")
```

How to search a shortest-step solutions?

25

Start

1	2	3
	4	6
7	5	8

Goal

1	2	3
4	5	6
7	8	

Limit = shortest steps

DFS path = ?

BFS path = ?

1	2	3
4	0	5
6	7	8

1	2	3
4	5	6
7	8	

DFS path = ?

BFS path = ?